CAD-bench: Benchmarking Language Models on Functional CAD Generation

Dhruv Saini
Independent Researcher
dhruv9saini@gmail.com

Abstract

Language-model agents increasingly operate computer-aided design (CAD) toolchains, yet executable code, rendered appearance, and coarse geometric similarity provide weak evidence of engineering correctness. Reliable CAD artifacts must also satisfy exact dimensions, mating interfaces, standards-like details, and assembly behavior. CAD-bench is an execution-based benchmark with 17 tasks organized into easy, medium, hard, and extreme tiers. The extreme tier contains threaded mating pairs and gear trains that require compatible interfaces and correct motion transmission. Each submission is executed, exported as geometry, and evaluated with task-specific dimensional, pose, reference-geometry, thread-profile, mating, and rigid-body checks. The same artifact interface supports one-shot CAD-code generation and agent harnesses that produce final STEP files. The strongest standalone model reaches 59.9% overall, but standalone systems average only 3.7% on the extreme tier; agent harnesses raise that average to 14.1%. Interfaces, standards-like details, and mechanisms therefore remain exceptionally difficult even when systems produce runnable CAD artifacts.

1 Introduction

Computer-aided design (CAD) is a central representation for physical product development. CAD artifacts encode geometry, dimensions, constraints, features, interfaces, and assemblies used in simulation, inspection, and manufacturing. Language-model agents can increasingly write code, operate tools, and iterate on execution feedback, making CAD generation a natural target for agentic automation.

Evaluation, however, remains a bottleneck. A generated CAD artifact can compile, render, and resemble a target object while still failing the engineering task. Common errors include shifted poses, incorrect hole placement, missing mating features, invalid thread profiles, gear collisions, intersecting bodies, and in general, assemblies that fail to transmit motion. These errors impact whether a generated artifact is usable at all.

Prior CAD-generation benchmarks establish important foundations. SketchGraphs represents CAD sketches as geometric constraint graphs [Seff et al., 2020]. CAD-as-Language and DeepCAD model CAD sketches or construction histories as serialized design programs [Ganin et al., 2021, Wu et al., 2021]. Fusion 360 Gallery provides human-authored design sequences and a programmatic reconstruction environment [Willis et al., 2021]. More recent systems address natural-language CAD generation, executable CAD code, and geometric validation. Text2CAD studies text-to-parametric-CAD generation from natural-language prompts [Khan et al., 2024]. CADPrompt pairs natural-language CAD prompts with expert CAD code and uses a visual verification loop [Alrashedy et al., 2024]. CAD-Coder and Text-to-CadQuery target text-to-CAD-code generation with executable feedback or geometric rewards [Guan et al., 2025, Xie and Ju, 2025]. CadEval, CADSmith, and CAD Arena evaluate rendering, geometry, validity, or programmatic measurements [Epoch AI, 2026,

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Track on Evaluations and Datasets.

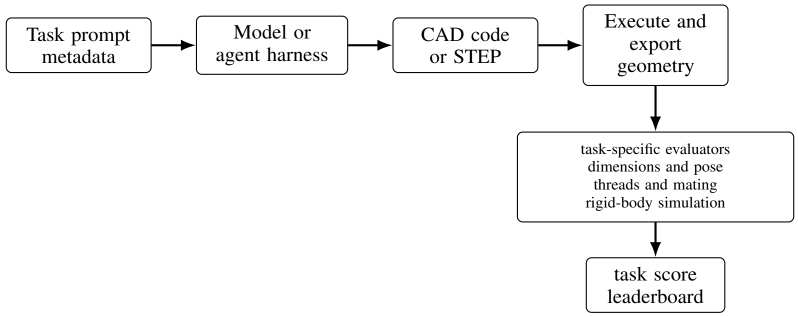

Figure 1: CAD-bench evaluation pipeline. A harness receives a task specification and produces CAD code or a STEP artifact. The runtime executes the submission, exports geometry, and applies task-specific checks for dimensions, pose, thread observables, mating behavior, and rigid-body function before aggregation.

Barkley et al., 2026, CAD Arena, 2026]. CAD-bench extends executable geometric evaluation to mechanically meaningful task requirements.

CAD-bench evaluates complete harnesses through a common output interface. A harness receives a task prompt and produces either CAD code or a STEP artifact. The benchmark executes the submission, exports geometry where needed, and applies task-specific evaluators. The 17 tasks progress from basic solids and feature-rich parts to standards-like mechanical components and extreme assembly problems. The extreme tier combines difficult shape synthesis with interface compatibility, thread mating, collision avoidance, and rigid-body motion.

Build success and engineering correctness diverge sharply on these tasks. The right-angle gearbox builds in 96.9% of reported agent attempts but averages only 4.0% on the full task score. Dimensional checks, interface checks, thread analysis, and rigid-body simulation reveal whether an executable artifact actually satisfies the prompt's mechanical requirements.

2 Related Work

CAD-bench builds on CAD generation, engineering-document understanding, and executable agent benchmarks.

CAD generation and CAD code benchmarks. Seff et al. [2020] introduce SketchGraphs, a large collection of CAD sketches represented as geometric constraint graphs. Ganin et al. [2021] formulate CAD sketch generation as a language modeling problem over serialized design programs, while Wu et al. [2021] model CAD construction histories as operation sequences. Willis et al. [2021] provide human-authored Fusion 360 design sequences and an environment for programmatic CAD reconstruction. More recently, Khan et al. [2024] introduce Text2CAD for generating sequential CAD models from natural-language prompts, and Alrashedy et al. [2024] introduce CADPrompt, a benchmark of natural-language CAD prompts paired with expert CAD code, along with a visual verification loop. CAD-Coder [Guan et al., 2025] and Text-to-CadQuery [Xie and Ju, 2025] move directly toward text-to-parametric-code generation, using executable CadQuery and geometric rewards such as Chamfer distance. CadEval [Epoch AI, 2026] evaluates text-to-CAD systems with rendering success, Chamfer and Hausdorff distances, and volume or consistency checks. CADSmith [Barkley et al., 2026] is especially close in spirit: it uses execution repair plus programmatic OpenCASCADE measurements and visual judging to refine CadQuery code. CAD Arena [CAD Arena, 2026] provides an emerging public comparison grid for 20 prompts, but its current reported metric is valid executable STL production.

These works establish CAD as a structured generation problem and show that executable geometric feedback matters. CAD-bench differs by scoring submitted artifacts against task-level mechanical requirements, including mating interfaces, thread observables, exact pose, and rigid-body function, rather than construction-sequence, rendering, or point-cloud similarity.

Engineering-document understanding. Doris et al. [2024] introduce DesignQA for evaluating multimodal understanding of engineering documentation. That work is complementary to CAD-bench: understanding drawings and manuals is part of the input side of engineering intelligence, while CAD-bench evaluates output-side artifact generation in a runnable CAD environment.

Executable benchmarks for agents. Recent work has shown the value of realistic, environment-grounded evaluation for agents. AgentBench [Liu et al., 2023], GAIA [Mialon et al., 2023], VisualWebArena [Koh et al., 2024], and OSWorld [Xie et al., 2024] benchmark agents in interactive environments rather than static text tasks. In software engineering, SWE-bench [Jimenez et al., 2023] and MLE-bench [Chan et al., 2024] show that end-to-end execution environments uncover failure modes that synthetic benchmarks miss. RE-Bench [Wijk et al., 2024] and PaperBench [Starace et al., 2025] extend this perspective to open-ended research engineering. CAD-bench adopts the same benchmark philosophy for CAD: models should be evaluated in the artifact-producing environment itself.

Benchmark methodology. BetterBench [Reuel et al., 2024] emphasizes best practices such as transparent reporting, replicability, and careful benchmark lifecycle management, while LiveBench [White et al., 2024] highlights the importance of contamination-resistant evaluation. CAD-bench follows these principles through executable scoring, explicit artifact traces, and a design that makes future task refreshes and extensions straightforward.

3 Motivation

CAD evaluation requires stronger checks than syntactic validity or visual plausibility. A model can generate valid Python, export a watertight mesh, or render an object with the correct silhouette while still producing the wrong dimensions, pose, feature placement, mating geometry, or assembly behavior. In mechanical design, these local errors are often decisive.

Surface-level geometric similarity is also incomplete. Many CAD tasks are defined by constrained interfaces rather than by global shape alone. A screw head with an incorrect socket, a thread that cannot mate, a gear train that collides, or a gearbox that fails to transmit motion may remain visually plausible under coarse rendering or point-cloud metrics. Such artifacts may look close to a reference but fail the engineering role specified by the prompt.

CAD agents also operate in executable tool environments, where they must synthesize parametric programs, reason over APIs and exchange formats, and satisfy exact toolchain semantics. CAD-bench is therefore designed around task-specific observables rather than visual complexity alone: visually simple tasks may require exact conventions or mating constraints, while visually complex tasks may be straightforward once decomposed into deterministic features.

4 Benchmark Design and Artifact Construction

4.1 Scope and task taxonomy

CAD-bench contains 17 tasks divided into four ascending difficulty levels: easy, medium, hard, and extreme. Table 1 summarizes the task structure.

The benchmark mixes single-part and multi-part tasks. The early tasks test basic parametric geometry, the middle tiers add feature density and standards-like details, and the extreme tasks require compatible interfaces and correct physical behavior across multiple parts.

4.2 Task packaging

Each task is stored as a prompt, metadata, expected values, and evaluator configuration. In the repository, tasks are specified through files such as `prompt.txt`, `task.toml`, and optional reference assets used for reproducibility. The public task format includes reference programs in `gold.py`, but those are benchmark-side reference implementations rather than the definition of the task itself. The expected metadata includes task ID, difficulty, evaluator, and task-specific expected values. Public task payloads can be loaded from a local mirror or from a Hugging Face repository.

Table 1: CAD-bench task taxonomy. The extreme tier concentrates the most demanding interface, assembly, and motion requirements.

Difficulty	Count	Example tasks	Main capability stress
Easy	3	cube, box, ring spacer	pose, units, simple solids
Medium	4	L-bracket, extrusion segment, slotted plate, hex nut blank	feature placement, boolean modeling, conventions
Hard	6	flange, bolted ring, stepped block, large slotted plate, spur gear, M3 socket-head screw	denser geometry, standards-like structure, gears, threads
Extreme	4	gearbox, custom threaded pair, compound gearbox, right-angle compound gearbox	assembly reasoning, interface compatibility, motion transmission

This packaging supports two important use cases: exact reproduction of published harness runs and evaluation of custom harnesses against the same task set and scoring logic.

The tasks are synthetic and engineering-grounded. Prompts specify concrete dimensions, poses, mating interfaces, thread conventions, and gearbox behavior; metadata records the expected observables; and evaluators test the submitted artifacts directly. Equivalent CAD construction histories can therefore satisfy the same task requirements.

4.3 Runtime and submission modes

The benchmark runtime supports two submission styles. In one-shot mode, code-oriented harnesses return CAD code inside a structured XML block, with the built-in implementation using Build123D [build123d contributors, 2026]. In step-based agent mode, an interactive harness writes the final CAD artifact to a designated STEP path inside the evaluation environment. In both cases, the runtime executes the submission in a containerized workspace and converts the result into a common scoring pipeline.

We report complete standalone model runs separately from complete agent runs. The agent interface remains part of the benchmark runtime, but agent rows are not mixed into the standalone model table because they use interactive tool feedback and a different wall-clock budget.

4.4 Scoring and verification

Each task returns normalized metrics including `build_success`, `task_score`, `overall_score`, and a benchmark-facing reward:

$$\texttt{reward} = 0.05 \cdot \texttt{build\_success} + 0.95 \cdot \texttt{overall\_score}. \tag{1}$$

At the benchmark level, CAD-bench first averages task `overall_score` within each difficulty bucket. Missing benchmark tasks are inserted as explicit zero-score rows for result aggregation. The final score is the category-weighted mean

$$S = \frac{1S_{\text{easy}} + 2S_{\text{medium}} + 3S_{\text{hard}} + 4S_{\text{extreme}}}{1+2+3+4}. \tag{2}$$

This means a zero on a difficulty bucket remains part of the denominator rather than silently dropping that category from the leaderboard. The 1:2:3:4 weighting gives the four extreme tasks appropriate influence alongside the 13 easier tasks, and every bucket is also reported separately. Section 7 reports an aggregation-sensitivity check.

The task evaluators are designed to match engineering reality. The benchmark uses:

- mesh- and part-level dimensional checks,
- pose and placement metrics,
- reference-geometry gates that downweight large deviations from the authored solution,
- thread-specific profile and pitch metrics for the custom threaded-pair task, and
- Blender-based rigid-body simulation for gearbox tasks.

Table 2: Representative scorer-calibration cases. Full is the CAD-bench `overall_score`. Build is a build-success-only ablation. Proxy is the available generic geometry proxy where one exists; gearbox rows are scored primarily through rigid-body simulation.

Task	Case	Full	Build	Proxy
Cube	independent valid cube	1.000	1.000	1.000
Cube	correct cube shifted off the required pose	0.000	1.000	0.000
Flange	independently constructed valid flange	1.000	1.000	1.000
Flange	bolt circle 1 mm too small	0.800	1.000	0.800
Flange	missing all four bolt holes	0.176	1.000	0.341
M3 screw	independently authored equivalent	1.000	1.000	1.000
M3 screw	threaded screw with missing socket	0.629	1.000	0.723
M3 screw	plain unthreaded shank	0.273	1.000	0.435
Gearbox	reference gear pair with correct motion	0.962	1.000	–
Gearbox	rigid bridge between axle holes	0.000	1.000	–
Gearbox	reference gear pair shifted off axles	0.000	1.000	–
Threaded pair	reference mating threaded pair	0.974	1.000	–
Threaded pair	plausible unthreaded bolt and nut	0.406	1.000	–
Compound gearbox	rigid three-axle bridge	0.000	1.000	–
Right-angle gearbox	intersecting reference-like assembly	0.000	1.000	–

Reference-geometry gates are used only as coarse sanity gates on selected tasks. They compare boolean volume differences between the submitted artifact and the reference artifact, then multiply the task score by a smooth penalty when the submitted shape has large missing or extra volume. The task-specific metrics remain responsible for dimensions, poses, threads, and function. This design allows ordinary alternative CAD construction histories, but intentionally reduces credit for artifacts that satisfy a few scalar checks while omitting major geometry.

This combination is central to the benchmark. A large part of CAD-bench's value is that it measures the difference between "the program ran" and "the artifact solved the task."

4.5 Scorer validation and ablations

Because CAD-bench uses task-specific evaluators, the release includes explicit scorer-calibration cases in `metadata/cad-bench-scorer-validation.json`. These cases cover reference submissions, independently authored valid submissions, minor defects, major defects, and nonbuild submissions across representative easy, hard-static, standards-like, threaded-pair, and extreme tasks. Table 2 summarizes the main cases.

The validation cases serve two purposes. First, they test invariance to implementation details by giving full credit to independently authored valid artifacts that are not copies of the corresponding `gold.py` programs. Second, they test sensitivity to engineering-relevant defects. Several defective artifacts build successfully but receive low full scores, which demonstrates why build success alone is an inadequate CAD metric. Generic geometry proxies also over-credit local-but-incomplete artifacts, such as a screw without a socket or a flange without bolt holes. For gearbox tasks, the decisive signal is rigid-body behavior: a rigid bridge with plausible axle holes builds and may have the approximately correct bounding box, but receives zero because it cannot transmit the required motion.

Worked M3 example. The M3 socket-head screw evaluator illustrates the scoring pattern. It first checks pose and dimensions: top of head at $z = 0$, full body in $z \leq 0$, head diameter and height, under-head length, and major diameter. It then checks contact geometry: socket depth, across-flats estimate, open radius at multiple depths, and sixfold harmonic signal. Finally it checks task-specific thread observables: helical signal, pitch, handedness, and approximate turns-to-seat. The final score is a weighted mean over generic pose, generic dimensions, generic contact, task thread, and task insertion modules. This is why the independently authored equivalent fixture receives full credit, a threaded screw without a socket receives partial credit, and an unthreaded cylinder receives only low residual credit despite building.

Table 3: Summary of complete CAD-bench baselines. Scores are category-weighted using Eq. (2). Full per-row results and run identifiers are reported in Appendix B.

Setting	Rows	Best overall	Mean easy	Mean hard	Mean extreme
Standalone models	19	0.599	0.930	0.585	0.037
Agent harnesses	32	0.677	0.906	0.761	0.141

5 Experimental Protocol

5.1 Model and harness matrix

The evaluation covers a mixed portfolio of closed and open providers. The result artifact contains 19 complete standalone model sweeps and 32 complete agent rows satisfying the validity criteria in Appendix B.3. Table 3 summarizes these rows, while Appendix B records the exact run identifiers and full row-level results. The standalone model set covers OpenAI, DeepSeek, Gemini, Vercel AI Gateway, and free OpenRouter rows.

The experiments use completed full-benchmark runs that cover all 17 tasks through model output and benchmark scoring. Run records retain provider quota failures, API outages, malformed transport responses, harness setup failures, and upload failures for accounting; model-quality comparisons use runs that reached CAD evaluation.

5.2 Reporting protocol

For each completed run, we report benchmark score, per-difficulty aggregates, wall-clock time, and estimated cost when the harness exposes enough usage and pricing data. When multiple complete runs exist for the same provider, model, access mode, and reasoning level, all complete rows in the result artifact are reported, and the exact run identifiers are recorded in Appendix B. We do not claim statistically significant pairwise rankings.

CAD-bench stores the artifacts needed for repeated-run reporting, allowing larger campaigns to replace single full-sweep rows with repeated-run aggregates.

6 Results

6.1 Overall performance

Table 3 summarizes the complete full-benchmark evaluations in the result artifact. Full leaderboards, run identifiers, wall-clock times, and cost estimates are reported in Appendix B; the main text focuses on aggregate conclusions because many configurations have only one complete sweep.

The best standalone model reaches 0.599 overall, and the best agent harness reaches 0.677. High easy-tier scores coexist with substantial room for improvement on the harder tiers.

The extreme tier is the clearest difficulty boundary. Completed standalone runs average 0.585 on hard tasks and 0.037 on extreme tasks; agent harnesses average 0.761 and 0.141, respectively. The right-angle gearbox makes this gap especially concrete: agent submissions build in 96.9% of attempts but average 4.0% on the full task score. Standalone submissions build the gearbox task in 63.2% of attempts and average 6.7%. The extreme tasks demand interface compatibility, thread mating, collision-free assemblies, and correct motion transmission beyond successful geometry export.

Agent harnesses generally outperform standalone one-shot generation, consistent with execution feedback helping repair dimensional, geometric, and interface errors. Their remaining failures concentrate in the extreme mechanisms even when the submitted artifacts build successfully.

6.2 Per-difficulty behavior

Across the 19 reported standalone model sweeps, the mean score by difficulty is 0.930 on easy, 0.753 on medium, 0.585 on hard, and 0.037 on extreme tasks. Across the 32 reported agent sweeps, the corresponding means are 0.906, 0.849, 0.761, and 0.141. Standards-like details and dense

Table 4: Per-task mean `overall_score` and build success across the complete reported rows. Standalone means use 19 model sweeps; agent means use 32 agent sweeps. This table is computed from the 867 task rows in the released result artifact, not from the difficulty aggregates alone.

Difficulty	Task	Standalone score	Standalone build	Agent score	Agent build
Easy	cube	0.895	0.947	0.938	0.969
Easy	box	0.947	1.000	0.844	0.844
Easy	ring spacer	0.947	1.000	0.938	0.938
Medium	L-bracket	0.632	0.842	0.906	0.969
Medium	extrusion	0.868	0.947	0.875	0.875
Medium	slotted plate	0.672	0.947	0.864	0.969
Medium	hex nut blank	0.842	0.895	0.750	0.906
Hard	flange	0.746	0.842	0.906	0.969
Hard	bolted ring	0.789	0.789	0.947	1.000
Hard	stepped block	0.812	0.947	1.000	1.000
Hard	large slotted plate	0.718	0.842	0.853	1.000
Hard	spur gear	0.237	0.526	0.345	0.844
Hard	M3 socket screw	0.207	0.526	0.512	0.875
Extreme	gearbox	0.067	0.632	0.316	0.844
Extreme	threaded pair	0.012	0.211	0.071	0.938
Extreme	compound gearbox	0.069	0.632	0.138	0.719
Extreme	right-angle gearbox	0.000	0.158	0.040	0.969

feature placement drive the medium-to-hard decline; interface compatibility and mechanism behavior produce the much larger decline at the extreme tier.

Reported aggregate scores decline monotonically from easy through extreme, with the largest drop at the extreme tier. Individual tasks still vary: several hard tasks are deterministic solids with many dimensions, while some lower-tier tasks are sensitive to exact orientation, alignment, or hole placement. Per-task results identify these local difficulty differences within the broader tier progression.

7 Discussion and Limitations

CAD-bench supports system diagnosis through executable CAD scoring, per-task traces, and controlled scorer cases. Many reported configurations have one full sweep, so the tables support aggregate capability and failure-mode analysis. Repeated-run statistics in Appendix B.2 confirm the hard-to-extreme difficulty gap for seven standalone configurations, and promoted leaderboard rows require repeated full sweeps under a declared aggregation rule.

The primary difficulty result is stable across aggregation rules. Under the complete result payload, the top standalone row remains Gemini 3.1 Pro offline under the weighted score, equal bucket averaging, and task-count weighting, with scores of 0.599, 0.727, and 0.709. For agent rows, weighted and equal-bucket aggregation place Codex GPT-5.4 mini medium web first at 0.677 and 0.781; task-count weighting places Pi GPT-5.4 high offline first at 0.772. Every aggregation shows a large performance gap between easy or static CAD and the extreme tier.

The 17 tasks provide dense coverage of primitives, feature placement, standards-like details, threaded interfaces, and simulated mechanisms. Broader CAD-agent assessment will require additional domains and task families. Appendix A describes the versioned release process: immutable public task bundles, private refresh tasks with canary hashes, contamination monitoring, and repeated-run promotion for leaderboard rows.

Task-specific scorers encode engineering assumptions for threaded pairs, socket-head screws, and gearboxes. A valid alternative solution can still miss an authored observable. The scorer-validation cases in Table 2 cover independently authored valid artifacts and controlled defects, while per-task traces and scorer components expose the basis of each aggregate score.

8 Conclusion

CAD-bench evaluates CAD generation through executable artifacts and mechanically meaningful checks. Current models and agent harnesses handle many simple solids but struggle with standards-like details, mating interfaces, and extreme mechanisms. The shared artifact protocol, task-specific evaluators, and rigid-body simulations make these failures measurable and give future systems concrete targets for producing assemblies and interfaces that work.

References

Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. Generating CAD Code with Vision-Language Models for 3D Designs. arXiv:2410.05340, 2024. URL: `https://arxiv.org/abs/2410.05340`.

Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. CADSmith: Multi-Agent CAD Generation with Programmatic Geometric Validation. arXiv:2603.26512, 2026. URL: `https://arxiv.org/abs/2603.26512`.

build123d contributors. build123d documentation. 2026. URL: `https://build123d.readthedocs.io/en/stable/index.html`.

CAD Arena. CAD Arena: Open Benchmark for AI-Generated Parametric CAD. 2026. URL: `https://cadarena.dev/`.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. arXiv:2410.07095, 2024. URL: `https://arxiv.org/abs/2410.07095`.

Anna C. Doris, Daniele Grandi, Ryan Tomich, Md Ferdous Alam, Mohammadmehdi Ataei, Hyunmin Cheong, and Faez Ahmed. DesignQA: A Multimodal Benchmark for Evaluating Large Language Models' Understanding of Engineering Documentation. arXiv:2404.07917, 2024. URL: `https://arxiv.org/abs/2404.07917`.

Epoch AI. CadEval. 2026. URL: `https://epoch.ai/benchmarks/cad-eval`.

Yaroslav Ganin, Sergey Bartunov, Yujia Li, Ethan Keller, and Stefano Saliceti. Computer-Aided Design as Language. arXiv:2105.02769, 2021. URL: `https://arxiv.org/abs/2105.02769`.

Yandong Guan, Xilin Wang, Xingxi Ming, Jing Zhang, Dong Xu, and Qian Yu. CAD-Coder: Text-to-CAD Generation with Chain-of-Thought and Geometric Reward. arXiv:2505.19713, 2025. URL: `https://arxiv.org/abs/2505.19713`.

Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL: `https://openreview.net/forum?id=5k9XeHIK3L`.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023. URL: `https://arxiv.org/abs/2310.06770`.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. arXiv:2401.13649, 2024. URL: `https://arxiv.org/abs/2401.13649`.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688, 2023. URL: `https://arxiv.org/abs/2308.03688`.

Gregoire Mialon, Clementine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for General AI Assistants. arXiv:2311.12983, 2023. URL: `https://arxiv.org/abs/2311.12983`.

Anka Reuel, Amelia Hardy, Chandler Smith, Max Lamparth, Malcolm Hardy, and Mykel J. Kochenderfer. BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices. arXiv:2411.12990, 2024. URL: https://arxiv.org/abs/2411.12990.

Ari Seff, Yaniv Ovadia, Wenda Zhou, and Ryan P. Adams. SketchGraphs: A Large-Scale Dataset for Modeling Relational Geometry in Computer-Aided Design. arXiv:2007.08506, 2020. URL: https://arxiv.org/abs/2007.08506.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI's Ability to Replicate AI Research. arXiv:2504.01848, 2025. URL: https://arxiv.org/abs/2504.01848.

Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. LiveBench: A Challenging, Contamination-Free LLM Benchmark. arXiv:2406.19314, 2024. URL: https://arxiv.org/abs/2406.19314.

Haoyang Xie and Feng Ju. Text-to-CadQuery: A New Paradigm for CAD Generation with Scalable Large Model Capabilities. arXiv:2505.06507, 2025. URL: https://arxiv.org/abs/2505.06507.

Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 Gallery: A Dataset and Environment for Programmatic CAD Construction from Human Design Sequences. *ACM Transactions on Graphics*, 40(4), 2021. URL: https://arxiv.org/abs/2010.02392.

Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. arXiv:2411.15114, 2024. URL: https://arxiv.org/abs/2411.15114.

Rundi Wu, Chang Xiao, and Changxi Zheng. DeepCAD: A Deep Generative Network for Computer-Aided Design Models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021. URL: https://arxiv.org/abs/2105.09492.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024. URL: https://arxiv.org/abs/2404.07972.

A Benchmark lifecycle, release policy, and broader impacts

CAD-bench v0 is a public, reproducible task set with exact run artifacts and a versioned release lifecycle. Four controls govern updates.

- **Frozen public splits.** Published task bundles are immutable and identified by manifest hashes and dataset revisions.
- **Private refresh tasks.** New extreme mechanism and standards-like tasks can be staged privately, with salted bundle-hash canaries exported before evaluation to establish that the tasks existed before result collection.
- **Versioned leaderboards.** Rows are reported against an explicit task-set version; old rows are not silently mixed with refreshed tasks.
- **Repeated-run promotion.** A configuration moves from diagnostic reporting to a primary leaderboard row only after independent full sweeps are available with a declared aggregation rule.

Task manifests record per-file hashes, the release pins the task dataset revision, and the source artifact contains a canary-export command for private or unreleased tasks. The v0 results establish benchmark behavior and failure modes; repeated sweeps provide the basis for future pairwise ranking analysis.

CAD agents could provide several benefits: lowering the barrier to prototyping, supporting accessibility-focused device customization, improving engineering education, and reducing repetitive modeling work in low-volume manufacturing workflows.

The same capabilities also create risks. More capable CAD generation could accelerate the design of unsafe hardware, restricted components, or deceptive engineering artifacts. It may also encourage overtrust in automatically generated mechanical designs that have not received appropriate engineering review. Transparent, function-aware benchmarking is one mitigation: it makes it harder to confuse runnable CAD code with mechanically correct design.

B Reproducibility and Reporting Details

B.1 Raw complete result rows

Tables 5–7 record the exact complete rows summarized in Table 3. Scores are reported as percentages for readability. The `run` column is the run identifier corresponding to the stored report artifact. Partial runs and infrastructure-failed rows are excluded from model-quality comparisons.

Table 5: Raw complete standalone model rows.

Rank	Harness	Overall	Easy	Med.	Hard	Extr.	Min.	Cost	Run
1	`gemini/gemini-3.1-pro-preview-offline-none`	59.9	100.0	96.8	70.8	23.2	25.4	5.8625	`gemini__gemini-3.1-pro-preview-offline-none_20260428T194054Z_2313253`
2	`openai/gpt-5.4-pro-web-med`	53.9	100.0	96.8	78.5	2.5	325.7	37.0405	`openai__gpt-5.4-pro-web-med_20260423T122643Z_1626647`
3	`vercel/alibaba/qwen-3.6-max-preview-offline-none`	52.8	100.0	96.8	76.2	1.5	42.8	3.2293	`vercel__alibaba__qwen-3.6-max-preview-offline-none_20260430T003706Z_2473332`
4	`gemini/gemini-3-flash-preview-offline-none`	52.6	100.0	96.8	77.4	0.0	21.1	1.3596	`gemini__gemini-3-flash-preview-offline-none_20260428T121645Z_2280737`
5	`openrouter/tencent/hy3-preview:free-offline-none`	51.1	100.0	96.8	60.8	8.7	51.4	–	`openrouter__tencent__hy3-preview_free-offline-none_20260428T035110Z_2240039`
6	`openai/gpt-5.3-codex-spark-offline-med`	50.4	100.0	96.8	70.0	0.0	6.1	4.9685	`openai__gpt-5.3-codex-spark-offline-med_20260427T023421Z_2115684`
7	`vercel/alibaba/qwen3.6-27b-offline-none`	50.0	100.0	96.8	68.9	0.0	58.6	1.5128	`vercel__alibaba__qwen3.6-27b-offline-none_20260430T003707Z_2473345`
8	`openai/gpt-5.3-codex-spark-offline-xhigh`	48.9	100.0	96.8	65.0	0.0	6.0	5.4877	`openai__gpt-5.3-codex-spark-offline-xhigh_20260427T024838Z_2122296`
9	`openai/gpt-5.5-offline-med`	48.7	100.0	71.8	65.0	12.0	9.8	8.6697	`openai_codex__gpt-5.5-offline-med_20260426T055852Z_2010916`
10	`openai/gpt-5.5-offline-low`	47.5	100.0	71.8	76.3	0.6	10.1	9.7384	`openai_codex__gpt-5.5-offline-low_20260426T054800Z_2000292`
11	`deepseek/deepseek-v4-pro-offline-none`	47.5	100.0	59.4	68.0	12.9	55.7	3.6274	`deepseek__deepseek-v4-pro-offline-none_20260426T042645Z_1958729`
12	`openai/gpt-5.5-offline-high`	45.6	66.7	71.8	82.1	0.0	8.9	7.8075	`openai_codex__gpt-5.5-offline-high_20260426T061252Z_2019625`
13	`openai/gpt-5.5-offline-xhigh`	45.6	100.0	71.8	59.3	8.7	9.9	9.2523	`openai_codex__gpt-5.5-offline-xhigh_20260426T062226Z_2023449`
14	`openai/gpt-5.3-codex-spark-offline-high`	40.4	100.0	96.8	36.9	0.0	6.1	5.2469	`openai__gpt-5.3-codex-spark-offline-high_20260427T024046Z_2118961`
15	`deepseek/deepseek-v4-flash-offline-none`	39.5	100.0	64.3	55.5	0.0	18.1	0.2778	`deepseek__deepseek-v4-flash-offline-none_20260426T052907Z_1985868`
16	`gemini/gemini-3.1-flash-lite-preview-offline-none`	34.2	100.0	75.0	30.8	0.0	19.2	0.1993	`gemini__gemini-3.1-flash-lite-preview-offline-none_20260428T050032Z_2249465`
17	`openai/gpt-5.3-codex-spark-offline-low`	33.9	100.0	71.8	31.7	0.0	5.8	4.2688	`openai__gpt-5.3-codex-spark-offline-low_20260427T022715Z_2111735`
18	`openrouter/inclusionai/ling-2.6-1t:free-offline-none`	13.9	33.3	2.7	33.3	0.0	12.8	–	`openrouter__inclusionai__ling-2.6-1t_free-offline-none_20260428T033043Z_2234914`
19	`gemini/gemma-3-27b-it-offline_nodocs-none`	8.1	66.7	0.0	4.6	0.0	3.7	–	`gemini__gemma-3-27b-it-offline_nodocs-none_20260430T022138Z_2493048`

B.2 Repeated-run statistical check

The result set reports complete full 17-task sweeps for every row, but many configurations have only one independent sweep. Those rows are therefore not used for pairwise model-ranking significance claims. To test the main qualitative difficulty claim under repeated sampling, the source artifact includes 47 complete local reports, with repeated full-sweep statistics for seven standalone configurations, summarized in `metadata/cad-bench-repeat-stats.json`. Additional attempted repeats across the reported standalone and agent matrix were excluded when provider quota, authentication, unavailable-model, or timeout failures prevented a complete 17-task report.

Table 6: Raw complete agent rows.

Rank	Agent	Harness	Overall	Easy	Med.	Hard	Extr.	Min.	Cost	Run
1	Codex	codex/gpt-5.4-mini-web-med	67.7	100.0	96.8	78.7	36.8	234.0	2.4446	codex__gpt-5.4-mini-web-med_20260420T000623Z_676664
2	Pi	pi/gpt-5.4-offline-high	67.1	100.0	96.8	82.9	32.1	618.8	4.6416	pi__gpt-5.4-offline-high_20260422T092300Z_1340452
3	Codex	codex/gpt-5.4-offline-high	65.3	66.7	71.8	83.6	47.9	245.2	8.2311	codex__gpt-5.4-offline-high_20260419T190624Z_603644
4	Pi	pi/gpt-5.4-offline-xhigh	64.7	100.0	96.8	88.0	22.3	418.3	4.8298	pi__gpt-5.4-offline-xhigh_20260421T075050Z_1041869
5	Pi	pi/gpt-5.4-web-high	64.6	100.0	96.8	82.9	25.9	281.6	4.5650	pi__gpt-5.4-web-high_20260422T092300Z_1340450
6	Pi	pi/gpt-5.4-mini-web-xhigh	61.0	100.0	100.0	84.5	14.2	240.2	2.4442	pi__gpt-5.4-mini-web-xhigh_20260422T092301Z_1340454
7	Pi	pi/gpt-5.4-mini-offline-high	60.3	100.0	75.0	80.3	28.0	169.0	3.5843	pi__gpt-5.4-mini-offline-high_20260422T151655Z_1441926
8	Pi	pi/gpt-5.4-offline-med	59.9	100.0	96.8	76.2	19.3	304.2	1.9074	pi__gpt-5.4-offline-med_20260421T075052Z_1041872
9	Codex	codex/gpt-5.4-web-low	59.1	100.0	96.8	79.4	14.8	279.8	6.3453	codex__gpt-5.4-web-low_20260419T190502Z_603630
10	Codex	codex/gpt-5.4-web-med	59.1	100.0	71.8	78.0	28.2	260.7	5.7144	codex__gpt-5.4-web-med_20260419T190540Z_603629
11	Pi	pi/gpt-5.4-mini-offline-xhigh	59.0	100.0	96.8	83.5	11.4	282.4	3.4776	pi__gpt-5.4-mini-offline-xhigh_20260422T121627Z_1402824
12	Codex	codex/gpt-5.4-mini-offline-med	57.8	100.0	96.8	52.5	31.7	157.2	3.3316	codex__gpt-5.4-mini-offline-med_20260420T004107Z_683812
13	Codex	codex/gpt-5.4-web-xhigh	56.9	100.0	96.8	89.7	1.7	396.2	12.7137	codex__gpt-5.4-web-xhigh_20260421T075051Z_1041874
14	Codex	codex/gpt-5.4-mini-offline-xhigh	56.9	100.0	96.8	77.8	10.5	120.9	4.0830	codex__gpt-5.4-mini-offline-xhigh_20260422T205507Z_1497871
15	Pi	pi/gpt-5.4-web-xhigh	56.0	100.0	71.8	82.9	16.9	419.3	4.7572	pi__gpt-5.4-web-xhigh_20260421T075052Z_1041870
16	Pi	pi/gpt-5.4-mini-web-med	54.5	100.0	96.8	81.8	1.5	191.2	1.6381	pi__gpt-5.4-mini-web-med_20260422T092300Z_1340457

Table 7: Raw complete agent rows, continued.

Rank	Agent	Harness	Overall	Easy	Med.	Hard	Extr.	Min.	Cost	Run
17	Pi	pi/gpt-5.4-mini-web-high	54.0	100.0	96.8	75.7	4.7	130.5	2.5139	pi__gpt-5.4-mini-web-high_20260422T183318Z_1468372
18	Codex	codex/gpt-5.4-mini-web-high	53.8	100.0	96.8	78.3	2.3	389.9	3.6274	codex__gpt-5.4-mini-web-high_20260420T235625Z_845623
19	Codex	codex/gpt-5.4-offline-xhigh	53.5	33.3	71.8	76.2	32.3	289.6	11.4765	codex__gpt-5.4-offline-xhigh_20260419T190535Z_603631
20	Pi	pi/gpt-5.4-offline-low	53.3	100.0	75.0	84.4	7.6	171.2	1.5744	pi__gpt-5.4-offline-low_20260419T061026Z_383966
21	Pi	pi/gpt-5.4-web-med	53.0	100.0	71.8	80.1	11.6	366.2	2.1871	pi__gpt-5.4-web-med_20260421T075053Z_1041868
22	Pi	pi/gpt-5.4-web-low	52.6	100.0	96.8	76.2	0.9	65.1	1.1919	pi__gpt-5.4-web-low_20260422T205357Z_1497569
23	Codex	codex/gpt-5.4-web-high	52.1	66.7	96.8	80.1	5.0	300.5	7.9083	codex__gpt-5.4-web-high_20260419T190546Z_603612
24	Codex	codex/gpt-5.4-mini-offline-high	51.3	100.0	96.8	71.1	1.5	169.1	4.4820	codex__gpt-5.4-mini-offline-high_20260420T004018Z_683339
25	Codex	codex/gpt-5.4-mini-web-xhigh	51.1	100.0	71.8	86.9	1.6	416.0	5.9764	codex__gpt-5.4-mini-web-xhigh_20260421T075051Z_1041875
26	Pi	pi/gpt-5.4-mini-offline-med	49.7	100.0	96.8	64.5	2.4	241.8	2.4102	pi__gpt-5.4-mini-offline-med_20260422T094713Z_1351194
27	Pi	pi/gpt-5.4-mini-web-low	48.1	100.0	75.0	77.1	0.0	189.4	0.5317	pi__gpt-5.4-mini-web-low_20260419T064631Z_413438
28	Codex	codex/gpt-5.4-mini-offline-low	48.1	100.0	100.0	57.5	2.2	111.3	1.6688	codex__gpt-5.4-mini-offline-low_20260420T014421Z_709021
29	Codex	codex/gpt-5.4-offline-low	46.8	33.3	46.8	88.4	18.9	181.1	4.4749	codex__gpt-5.4-offline-low_20260419T190806Z_603646
30	Pi	pi/gpt-5.4-mini-offline-low	37.9	100.0	71.8	38.5	4.9	132.1	0.5499	pi__gpt-5.4-mini-offline-low_20260422T100629Z_1359611
31	Codex	codex/gpt-5.4-offline-med	36.2	33.3	46.8	61.6	12.6	260.4	8.0969	codex__gpt-5.4-offline-med_20260419T190604Z_603645
32	Codex	codex/gpt-5.4-mini-web-low	32.9	66.7	50.0	54.1	0.0	246.5	1.8227	codex__gpt-5.4-mini-web-low_20260420T000933Z_677450

Across six `openai/gpt-5.3-codex-spark-offline-med` sweeps, the benchmark score is 0.471 with bootstrap 95% CI $[0.453, 0.490]$. The easy bucket is stable at 1.000, medium is 0.973 with CI $[0.968, 0.984]$, hard is 0.572 with CI $[0.507, 0.643]$, and extreme is 0.011 with CI $[0.000, 0.033]$. A paired exact two-sided sign-flip test over full-sweep difficulty aggregates gives $p = 0.03125$ for hard score minus extreme score, with mean difference 0.561 and bootstrap 95% CI $[0.486, 0.641]$.

Across the four six-sweep `openai/gpt-5.5-offline-*` repeated checks, benchmark means are 0.477 for low with CI $[0.455, 0.492]$, 0.444 for medium with CI $[0.391, 0.503]$, 0.505 for high with CI $[0.454, 0.563]$, and 0.573 for xhigh with CI $[0.528, 0.612]$. Their hard-minus-extreme mean differences are 0.701, 0.483, 0.695, and 0.694, respectively, and each exact sign-flip test gives $p = 0.03125$. The xhigh row has the strongest repeated GPT-5.5 mean here; these runs establish a within-configuration hard-to-extreme difficulty contrast while the model-level tables remain diagnostic baselines.

Across six `openrouter/tencent/hy3-preview:free-offline-none` sweeps, the benchmark score is 0.518 with CI $[0.471, 0.557]$; easy is 1.000, medium is 0.823 with CI $[0.729, 0.917]$, hard is 0.664 with CI $[0.585, 0.737]$, and extreme is 0.135 with CI $[0.095, 0.176]$. The hard-minus-extreme sign-flip test gives $p = 0.03125$, mean difference 0.529, and CI $[0.472, 0.594]$. Across five `openrouter/inclusionai/ling-2.6-1t:free-offline-none` sweeps, the benchmark score is 0.154 with CI $[0.080, 0.236]$; hard-minus-extreme remains positive at 0.227 with CI $[0.084, 0.407]$, but the exact sign-flip test is $p = 0.125$ at $n = 5$. Across multiple standalone harnesses, the repeated sweeps consistently identify the extreme tier as the largest remaining difficulty boundary.

The repeated-run artifact also includes the `metadata/cad-bench-repeat-check.json` two-run sanity record for traceability. Repeated-sweep reports use the following format:

- number of independent full sweeps per configuration,
- central estimate (mean or median),
- uncertainty interval (e.g., standard deviation or bootstrap 95% CI),
- policy for selecting primary leaderboard rows when multiple repetitions exist.

The analysis therefore avoids pairwise significance claims and uses the complete sweeps as baseline diagnostics.

B.3 Run validity criteria

A run is eligible for a complete-results table if it satisfies all of the following conditions:

- it uses a standalone model harness or an agent harness,
- it attempts the full 17-task benchmark under one fixed provider, model, access mode, and reasoning setting,
- each task reaches model-output capture, STEP export when applicable, and benchmark scoring, and
- failures, if any, are model-output or scored-artifact failures rather than provider quota, API outage, harness setup, or artifact-upload failures.

Provider and infrastructure failures are retained in the experiment logs for auditability, but they are not assigned model-quality scores because they do not evaluate the submitted model on CAD generation.

B.4 Compute and environment reporting

The benchmark runtime uses Dockerized environments, Python 3.11, NumPy, trimesh, and Blender for the rigid-body simulation tasks, together with Build123D in the code-oriented reference and harness pipeline. The reported local runs used the following environment:

- host hardware: Intel Core i5-7300U CPU, 15 GiB RAM, CPU-only scoring except for Blender's local simulation/rendering stack,
- software versions: Docker 29.4.0, Blender 5.1.1, uv 0.11.7, project Python 3.11 environment from `uv.lock`,
- standalone model wall-clock range in the reported rows: 3.7–325.7 minutes,
- agent wall-clock range in the reported rows: 65.1–618.8 minutes,
- failed-run accounting: provider quota failures are kept in logs but excluded from model-quality comparisons.

B.5 Code, data, and asset availability

The benchmark runtime, scoring code, and harness interface are available in the source artifact. Public task payloads and result artifacts include every complete run reported in this manuscript.

The artifact includes a no-API smoke path for package integrity and scorer calibration. Full model sweeps require provider credentials, but scorer validation and artifact consistency do not.

The release pointers are:

- task dataset revision: `20ad0b586b70ac2c5e6fe2b501386cf26a887137`,
- result artifact: `results/cad-bench-reported-results.json`,
- scorer validation artifact: `results/cad-bench-scorer-validation.json`,
- run artifact bundles: stored with the released result records.

The artifact includes Croissant metadata with Responsible AI fields describing intended use, limitations, task authoring, and safety-relevant non-use cases.

B.6 Licenses and upstream assets

Table 8 documents the main upstream software and data assets used by the benchmark runtime and dataset.

Table 8: Primary upstream software assets used by CAD-bench.

Asset	License / terms	Role in benchmark
Build123D	Apache-2.0	reference implementation and code-harness dependency, not the benchmark definition itself
Blender	GNU GPL	rigid-body simulation and rendering for extreme gearbox evaluation
NumPy	BSD-style license	numeric processing in evaluators and utilities
trimesh	MIT	mesh conversion and geometric analysis
Repository code	MIT	benchmark runtime, harnesses, and scoring stack
Public task dataset	MIT	hosted task prompts, metadata, reference programs, and authored benchmark assets
Authored CAD fixtures	MIT	synthetic task-side fixtures, including the M3 socket-head screw equivalent fixture
Closed model APIs	provider terms	external inference services used for benchmarked models

The benchmark tasks and authored fixtures are synthetic benchmark assets rather than redistributed proprietary CAD files.